

Contents

1	Implementation Plan: Enable PML (absorbing boundaries) for the SAFS spatial dynamic-rupture run	1
1.1	Overview	1
1.2	Part A — Theory of the PML as implemented in this codebase	2
1.3	Part B — Current state audit (what’s wired, what’s not)	5
1.4	Part C — Domain sizing analysis (Phase 0 — analysis only, no code)	5
1.5	Constraints	7
1.6	Phase 1 — Make PMLayer free-surface-safe (per-half-face damping) + tests	7
1.7	Phase 2 — Construct and wire the PML in seas_spatial_dyn_driver	8
1.8	Phase 3 — Validate PML effectiveness on the SAFS problem	10
1.9	Phase 4 (optional) — Production integration & remeshed deep-box variant	12
1.10	Testing Strategy	12
1.11	Risk Assessment	12
1.12	Summary of file touches (for the implementer)	13

1 Implementation Plan: Enable PML (absorbing boundaries) for the SAFS spatial dynamic-rupture run

Date: 2026-05-27 **Author:** planning agent (code-plan) **Scope:** wire the existing convolutional-PML machinery into seas_spatial_dyn_driver, make it free-surface-safe, size it for the SAFS box, and define how to verify it works.

1.1 Overview

The SAFS dynamic-rupture box is reflection-free only for $t_{\text{reflect}} = \min_{\text{box_dim}} / c_p = 41.6 \text{ km} / 5996 \text{ m/s} \approx 6.94 \text{ s}$, yet the production runs use $t_{\text{final}} = 100 \text{ s}$. Reflected waves therefore re-load the fault after $\sim 7 \text{ s}$ and contaminate everything later — including the slip-weakening (LSW) V_{max} departure at $t \approx 28 \text{ s}$ that we currently cannot attribute to a friction instability vs. reflected-wave reloading (jobs 7752380/7753174). A working PML removes that confound.

The codebase **already implements** an unsplit convolutional PML (dynamic/pml_layer.{hpp,cpp}) and the WaveOperator damping hooks (SetPML, ApplyPMLDamping, predictor + corrector application of $-d(x) \cdot D \cdot (Q - Q_{\text{bg}})$), and there are 4 passing unit tests (tests/unit/test_pml.cpp). What is missing: (a) **no driver ever constructs a PMLayer or calls wave.SetPML()** — `--pml` only suppresses the reflection warning; (b) the damping profile is **symmetric in z**, which would wrongly damp the **free surface** at $z = 0$; (c) there are **no config knobs** for thickness / target reflection / which faces. This plan closes those three gaps and validates the result.

Key result of the sizing analysis (Phase 0 below): no remesh / no box extension is required. The existing 50/50/25 km pads already host a 12–15 km PML shell on the five absorbing faces with $\geq 10 \text{ km}$ of buffer between the fault and the PML inner edge.

1.2 Part A – Theory of the PML as implemented in this codebase

A.1 Governing equations (what the PML damps)

The bulk solver advances the 3-D **first-order velocity–stress** elastodynamic system. In physical (tensor) form, with velocity $\mathbf{v}$, stress σ , density ρ and Lamé parameters λ, μ :

$$\rho \partial_t \mathbf{v} = \nabla \cdot \sigma, \quad \partial_t \sigma = \lambda (\nabla \cdot \mathbf{v}) \mathbf{I} + \mu (\nabla \mathbf{v} + (\nabla \mathbf{v})^\top).$$

The code stacks the six independent stresses and three velocities into one state vector $\mathbf{Q} \in \mathbb{R}^9$ (dynamic/wave_state.hpp, NUM_STATE = 9):

$$\mathbf{Q} = \left(\underbrace{\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}}_{6 \text{ stress components}}, \underbrace{v_x, v_y, v_z}_{3 \text{ velocities}} \right)^\top.$$

Both PDEs above are linear, so they collapse to a single flux-Jacobian form with constant 9×9 matrices $\mathbf{A}_k = \mathbf{A}_k(\lambda, \mu, \rho)$:

$$\partial_t \mathbf{Q} = \sum_{k \in \{x, y, z\}} \mathbf{A}_k \partial_{x_k} \mathbf{Q} \equiv \mathcal{L}(\mathbf{Q}),$$

where $\mathcal{L}$ is exactly the elastic right-hand side the DG operator assembles in wave_operator.inl. **The PML adds one local (zeroth-order, no new derivatives) damping term** that pulls $\mathbf{Q}$ back to a reference state inside the absorbing shell:

$$\partial_t \mathbf{Q} = \mathcal{L}(\mathbf{Q}) - \mathbf{d}(\mathbf{x}) \odot (\mathbf{Q} - \mathbf{Q}_{\text{bg}}) \quad (\text{Eq. 16})$$

Here $\odot$ is the componentwise (Hadamard) product and $\mathbf{d}(\mathbf{x}) = (d_1, \dots, d_9)$ is a **per-component damping-rate** vector (units s^{-1}), defined in A.3–A.4. In the interior $\mathbf{d} \equiv \mathbf{0}$, recovering $\partial_t \mathbf{Q} = \mathcal{L}(\mathbf{Q})$ exactly. This is the **unsplit convolutional PML (CPML)** of Komatitsch & Martin (2007), here in its memory-variable-free, directional approximation: the absorption is applied directly through $\mathbf{d}$ and the selection matrices of A.4, with no auxiliary convolution variables (pml_layer.hpp:33).

A.2 Why the target is $\mathbf{Q}_{\text{bg}}$, not 0 (the total- $\mathbf{Q}$ / fluctuation form)

The SAFS solver runs in **total- $\mathbf{Q}$** mode: $\mathbf{Q}$ carries the static pre-stress background $\mathbf{Q}_{\text{bg}}$ (set via WaveOperator::SetAbsorbingBackground, driver:1325; stored as bulk_bg_[NUM_STATE], gated by has_bulk_bg_, wave_operator.hpp:932–933). Equation (16) damps the **fluctuation** $\mathbf{Q} - \mathbf{Q}_{\text{bg}}$ — i.e. the radiated wave — driving $\mathbf{Q} \rightarrow \mathbf{Q}_{\text{bg}}$, *not* $\mathbf{Q} \rightarrow \mathbf{0}$. Per scalar component c :

$$\dot{Q}_c = [\mathcal{L}(\mathbf{Q})]_c - d_c(\mathbf{x}) (Q_c - Q_{\text{bg},c}).$$

“damping $\mathbf{Q}$ toward zero would erode the pre-stress tensor in the PML region and create a stress-gradient artefact at the PML/interior interface” — wave_operator.inl:5778–

5781.

ApplyPMLDamping (wave_operator.inl:5729) asserts has_bulk_bg_ and subtracts $Q_{\text{bg},c}$ at each quadrature point before damping. **Prerequisite already satisfied** for the SAFS driver.

A.3 The damping rate: profile and peak value

Inside the shell, d_c is built from three **directional** rates d_x, d_y, d_z (A.4). Each is a cubic ramp of the *penetration depth* s_ξ — how far a point has entered the layer measured from its inner edge. For the x -layers of a box $[x_{\min}, x_{\max}]$ with thickness L (pml_layer.cpp:61–92):

$$s_x(x) = \max\left(0, (x_{\min} + L) - x, x - (x_{\max} - L)\right)$$

— zero in the interior, rising to L at either wall (the two opposite layers never overlap, so the max just picks whichever side the point is in). s_y, s_z are analogous. The directional rate ramps **cubically** (pml_layer.cpp:50–56):

$$d_\xi(\xi) = d_{\max} \left(\frac{s_\xi}{L}\right)^3, \quad \xi \in \{x, y, z\},$$

with peak value, **as coded** (pml_layer.cpp:44):

$$d_{\max} = \frac{3 c_p}{2 L} \ln \frac{1}{R_0}. \quad (\text{Eq. 17})$$

c_p = P-wave speed (the fastest wave $\rightarrow$ conservative), L = layer thickness, $R_0 \in (0, 1)$ = target reflection coefficient.

Where Eq. 17 comes from (and a precision caveat). A plane wave at incidence angle θ that crosses the layer, reflects off the outer wall, and returns is attenuated by the round-trip optical-depth factor

$$R(\theta) = \exp\left(-\frac{2 \cos \theta}{c_p} \int_0^L d(s) \, ds\right).$$

For a power-law profile $d(s) = d_{\max} (s/L)^n$ the integral is $\int_0^L d \, ds = \frac{d_{\max} L}{n+1}$, so requiring $R(0) = R_0$ at normal incidence gives the **consistent grading constant**

$$d_{\max} = \frac{(n+1) c_p}{2 L} \ln \frac{1}{R_0}.$$

The code uses a **cubic** profile ($n = 3$) but the $n = 2$ prefactor $\frac{n+1}{2} = \frac{3}{2}$. With a cubic profile the consistent constant is $\frac{4}{2} = 2$, i.e. $d_{\max} = \frac{2c_p}{L} \ln \frac{1}{R_0}$. Plugging the as-coded $\frac{3}{2}$ back into the round-trip integral, the layer actually realizes

$$R_{\text{eff}} = \exp\left(-\frac{2 d_{\text{max}} L}{(n+1) c_p}\right) \Big|_{n=3} = \exp\left(-\frac{3}{4} \ln \frac{1}{R_0}\right) = R_0^{3/4},$$

e.g. $R_0 = 10^{-3} \Rightarrow R_{\text{eff}} \approx 5.6 \times 10^{-3}$ — slightly *weaker* than the nominal label. This is not fatal (the absorption is still strong), but it is a real inconsistency: **Phase 2 should either switch the prefactor to $2c_p/L$ (true $n = 3$) or keep $\frac{3}{2}$ and document that the knob is R_{eff} , not R_0 .** Either way the Phase-3 metrics measure the *true* reflection, so this only affects how the R0 input is labeled.

A.4 Directional / corner damping (the selection vectors $\mathbf{D}^\xi$)

A given face should damp only the components that carry energy across it. The per-component rate sums the directional contributions (`wave_operator.inl:5790–5792`):

$$d_c(\mathbf{x}) = d_x(x) D_c^x + d_y(y) D_c^y + d_z(z) D_c^z, \quad c = 1, \dots, 9,$$

with binary selection vectors $\mathbf{D}^x, \mathbf{D}^y, \mathbf{D}^z \in \{0, 1\}^9$ (`pml_layer.cpp:24–26`):

$\mathbf{D}^\xi$	σ_{xx}	σ_{yy}	σ_{zz}	σ_{xy}	σ_{yz}	σ_{xz}	v_x	v_y	v_z
$\mathbf{D}^x$	1	0	0	1	0	1	1	0	0
$\mathbf{D}^y$	0	1	0	1	1	0	0	1	0
$\mathbf{D}^z$	0	0	1	0	1	1	0	0	1

Read a column as “which directions damp this component”: σ_{xy} is damped by both x and y ; v_z only by z . In an **edge or corner** where two/three layers overlap, the active d_ξ simply **add**, so a shared component relaxes at the summed rate (the “R-004 corner fix”, `pml_layer.hpp:31`). `ComputeDamping` (`pml_layer.cpp:61–92`) returns (d_x, d_y, d_z) .

A.5 Where the term enters the time integration

`ApplyPMLDamping(Q, rhs)` is invoked in **two** places, both gated on `pml_layer_ != nullptr`:

1. **Standard operator** `WaveOperator::Mult` — `wave_operator.inl:902–905` (`dQdt -= d·D·(Q–Q_bg)`).
2. **ADER corrector** — inline at `wave_operator.inl:5438–5490`, with the damping target scaled to `dt·Q_bg` because under the ADER corrector the background is itself time-integrated (I is the time-integrated state).

Note (`wave_operator.hpp:427`): PML is **intentionally not** in the ADER local predictor (a per-element extrapolation); it is applied in the corrector that actually advances the solution. The SAFS config runs `ader_order = 2`, so **the corrector path (5438) is the operative one** — this is the path to verify in Phase 3.

A.6 Quadrature integration of the damping

The damping is integrated elementwise on the same DG basis (`wave_operator.inl:5741–5800`). For element e with basis $\{\phi_i\}$, the contribution to the residual of component c at node i is

$$\text{rhs}_{c,i} = - \sum_q w_q d_c(\mathbf{x}_q) \phi_i(\mathbf{x}_q) \left(Q_c(\mathbf{x}_q) - Q_{\text{bg},c} \right), \quad Q_c(\mathbf{x}_q) = \sum_j \phi_j(\mathbf{x}_q) Q_{c,j},$$

where $w_q = \hat{w}_q |J(\mathbf{x}_q)|$ are the mapped quadrature weights and d_c is evaluated from the physical coordinate $\mathbf{x}_q$ via A.3–A.4. Elements with $d_x = d_y = d_z = 0$ at every quadrature point are skipped (if $(dx \leq 0 \ \&\& \ dy \leq 0 \ \&\& \ dz \leq 0)$ continue;), so the cost scales with the thin PML shell, not the whole mesh.

1.3 Part B – Current state audit (what’s wired, what’s not)

Component	Status	Evidence
PMLayer (profile, d_max, corner D)	Implemented	pml_layer.{hpp,cpp}
WaveOperator::SetPML / GetPML	Implemented	wave_operator.hpp:246–247
ApplyPMLDamping (predictor + corrector)	Implemented	wave_operator.inl:902, 5438, 5729
Total-Q background SetAbsorbingBackground	Wired in driver	spatial_dyn_driver.cpp:1325
Unit tests (normal/oblique/decay/corner)	Pass	tests/unit/test_pml.cpp (4 tests)
cfg.numerics.use_pml + --pml flag	Parsed, but inert	driver:532,624,757,973 — only gates the warning
PMLayer constructed + SetPML() called	MISSING	grep 'PMLayer(' finds only the definition; no driver instantiates it
Free-surface-safe z (bottom-only)	MISSING	pml_layer.cpp:84–91 damps z at <i>both</i> z_min and z_max; z_max = 0 is the free surface (attr 102)
Config knobs (thickness, R0, faces)	MISSING	spatial_friction.{hpp,cpp} has only use_pml (bool)

Boundary attributes (driver:840-843): fault = 101, free surface (top) = 102 (natural_attrs), bottom + sides = 103/104 (absorbing_attrs, currently 1st-order ABC). PML must cover the absorbing faces ($x_{\pm}$, $y_{\pm}$, **z-bottom**) and must **not** touch attr 102.

1.4 Part C – Domain sizing analysis (Phase 0 – analysis only, no code)

C.1 Measured geometry (500 m triq mesh, the one these jobs used)

Box : X[314840, 672330] Y[3642278, 3888985] Z[−41608, 0] (dx=357.5, dy=246.7, dz=41.6 km)

Fault : X[364840, 622330] Y[3692278, 3838985] Z[-16608, 0] (dx=257.5, dy=146.7, dz=16.6 km)
Buffer (fault → wall): +x 50, -x 50, +y 50, -y 50 km; bottom 25 km; top = free surface
Material: $\lambda = \mu = 3.2e10$ Pa, $\rho = 2670 \rightarrow c_p = 5996.2$ m/s, $c_s = 3461.9$ m/s
lc_far = 3000 m (far-field cell size, where the PML lives); fe order = 1; ader_order = 2

C.2 Why a bigger box is the wrong tool

To make the **no-PML** reflection-free window reach $t_{\text{final}} = 100$ s, the nearest wall must be $> c_p \cdot t_{\text{final}} / 2 \approx 300$ km from the active fault — i.e. ~ 600 km pads. Infeasible (the box is already $357 \times 247 \times 42$ km). PML is the correct tool: it absorbs at the existing wall instead of delaying the return.

C.3 PML thickness L_{pml}

Constraints: - **Enough cells to grade the cubic profile.** At $lc_{\text{far}} = 3000$ m, $L_{\text{pml}} = 12$ km $\rightarrow$ 4 cells, 15 km $\rightarrow$ 5 cells. Low-order DG wants ≥ 4 –5 element layers; 5 is the target. - **Fit inside the buffer without reaching the fault.** With $L_{\text{pml}} = 15$ km: inner edges at $x = 329.8 / 657.3$ km, $y = 3657.3 / 3874.0$ km, $z = -26.6$ km. Fault-to-PML clearance: $x \geq 35$ km, $y \geq 34$ km, **bottom 10 km** (tightest). With $L_{\text{pml}} = 12$ km: bottom clearance 13 km, sides ≥ 38 km.

Recommendation: $L_{\text{pml}} = 12$ km (4 far-field cells) as the default — keeps a 13 km bottom buffer. Offer 15 km (5 cells) as an override if Phase-3 metrics want more absorption. **No remesh needed** at either value.

C.4 Resulting d_{max} (Eq. 17, $c_p = 5996$ m/s)

L_{pml}	$R0 = 1e-3$	$R0 = 1e-4$
12 km	$d_{\text{max}} = 5.18 \text{ s}^{-1}$	6.90 s^{-1}
15 km	$d_{\text{max}} = 4.14 \text{ s}^{-1}$	5.52 s^{-1}

Default $R0 = 1e-3$ (matches PMLayer default and the unit tests). The damping timescale $1/d_{\text{max}} \approx 0.19$ – 0.24 s is $\ll$ the ~ 7 s reflection window, so an outgoing wave is attenuated by $\approx R0$ over a round trip through the layer.

C.5 Wavelength sanity check (is 12–15 km “thick enough”?)

- Body waves: rupture rise time $\sim D_c/V_{\text{peak}} \approx 2.0/10 = 0.2$ s $\rightarrow f \sim 5$ Hz $\rightarrow \lambda_s \approx 0.7$ km, $\lambda_p \approx 1.2$ km. A 12–15 km PML is **many** body wavelengths $\rightarrow$ body-wave reflections (the dominant ~ 7 s bottom/side contamination) are absorbed essentially to $R0$.
- Surface (Rayleigh) waves along the free surface to the x/y walls: longest periods ~ 0.1 – 1 Hz $\rightarrow \lambda \approx 3.5$ – 35 km. 12–15 km is ~ 0.4 – 4λ — adequate for the shorter-period content; the very-longest-period surface energy is the residual the Phase-3 metrics must quantify. (If insufficient, thicken the x/y PML or accept the 35–50 km buffer as the long-period guard.)

C.6 Do we extend the domain? – No (default). Optional later.

- **Default (Phases 1–3): carve the PML from the existing box.** No remesh; the outer 12 km shell of the current mesh becomes the sponge, replacing the leaky 1st-order ABC on attrs 103/104.

- **Optional (Phase 4, only if Phase-3 shows the 13 km bottom buffer is too thin):** remesh with `--pad-bottom 40000` (deepen the box by 15 km) so the bottom PML does not encroach on the sub-fault region. Sides/top unchanged. This is a parameter change to `run_z0cut_meshing.py`, not new code.

1.5 Constraints

- **Do not modify** the elastic RHS, the friction laws (`fault_face_flux.*`, `tpv205_friction.hpp`), `SetAbsorbingBackground`, or the TPV102/104/205 byte-exact regression paths. PML is additive and gated on `pml_layer_ != nullptr`.
- **Preserve the existing PMLayer ctor signature and unit-test behavior** — extend, don't break (`test_pml.cpp` constructs `PMLayer(xmin, xmax, thk, cp[, R, dirs])`).
- **Free surface (attr 102) must never be damped.** z-PML is bottom-only for SAFS.
- **No hardcoded magic numbers** in the driver — thickness/R0/faces come from config or are derived from the mesh `bbox + lc_far` (per project convention).
- Follow the dated-document convention (this file) and the SAFS “no-Tandem for dynamic work” rule (this is grounded in the repo's own PML/wave machinery).
- Lifetime: the PMLayer object must outlive the time loop (stored as a raw `pml_layer_` pointer).

1.6 Phase 1 – Make PMLayer free-surface-safe (per-half-face damping) + tests

Goal

PMLayer can damp any subset of the 6 half-faces, so the SAFS run damps $x_{\pm}$, $y_{\pm}$, z-min while leaving z-max (free surface) untouched — with the existing symmetric behavior preserved as the default.

Files to Modify

- `dynamic/pml_layer.hpp` — extend the interface with an optional per-half-face mask.
- `dynamic/pml_layer.cpp` — honor the mask in `ComputeDamping`.
- `tests/unit/test_pml.cpp` — add a free-surface test (below).

Detailed Requirements

1. Add a 6-bit half-face mask. Bit layout (LSB→MSB): `XL0=1`, `XHI=2`, `YL0=4`, `YHI=8`, `ZL0=16`, `ZHI=32`. Provide enum or static constexpr int named constants in PMLayer (e.g. `PMLayer::FaceXL0 ...`). Add static constexpr int `FaceAll = 0x3F`;
2. Extend the constructor with an **optional** trailing parameter, preserving the current signature:

```
PMLayer(const Vector &x_min, const Vector &x_max,
        real_t thickness, real_t cp,
        real_t target_R = 1e-3, int dirs = 7,
        int half_face_mask = -1); // -1 ⇒ derive from `dirs` (all faces of each enabled
```

When `half_face_mask == -1`, derive it from `dirs` so existing call sites are byte-identical: `XL0|XHI` if `dirs&1`, `YL0|YHI` if `dirs&2`, `ZL0|ZHI` if `dirs&4`. Store the resolved mask in a new

int face_mask_member.

3. In `ComputeDamping`, gate each half-side on its mask bit. The x-block becomes:

```
if (face_mask_ & FaceXLo) { real_t d = (x_min_(0)+L_pml_) - x; if (d>0) dx = DampingProfile  
if (face_mask_ & FaceXHi) { real_t d = x - (x_max_(0)-L_pml_); if (d>0) dx = std::max(dx, D
```

Likewise y (`FaceYLo/FaceYHi`) and z (`FaceZLo/FaceZHi`). The `dirs_` member may be removed in favor of `face_mask_` (update the ctor accordingly), but keep the `dirs` ctor *argument* for source compatibility.

4. Keep `DampingProfile`, `Dx/Dy/Dz`, `d_max_`, accessors unchanged.

Interfaces

- `PMLayer(const Vector&, const Vector&, real_t, real_t, real_t, int, int)` (new optional arg).
- `static constexpr int PMLayer::FaceXLo, FaceXHi, FaceYLo, FaceYHi, FaceZLo, FaceZHi, FaceAll;`
- `int GetFaceMask() const;`

Edge Cases to Handle

- `half_face_mask == -1` → exactly reproduce today’s symmetric behavior (regression-critical for the 4 existing tests).
- `half_face_mask == 0` → no damping anywhere (degenerate; allowed, useful for an “ABC-only” A/B without changing call sites).
- A point in a disabled half-face returns 0 for that direction even if geometrically inside the shell.

Acceptance Criteria

- ☐ The 4 existing `test_pml.cpp` tests pass **unchanged** (default mask = symmetric).
- ☐ New test `TestPMLFreeSurfaceTopUndamped`: cubic box, PML on `XL0|XHI|YL0|YHI|ZL0` (not `ZHI`); assert `ComputeDamping` returns `dz==0` for any point within `L_pml` of `z_max`, and `dz>0` within `L_pml` of `z_min`.
- ☐ New test `TestPMLFreeSurfacePulseStable`: a P-pulse reflecting off the undamped `z_max` free surface stays finite for 500 steps (no NaN) and the `z_min` side absorbs (interior energy decays).
- ☐ make test green.

Dependencies

- Depends on: nothing. Required by: Phase 2.

1.7 Phase 2 – Construct and wire the PML in `seas_spatial_dyn_driver`

Goal

When PML is enabled, the driver builds a `PMLayer` from the mesh bounding box and config, calls `wave.SetPML(&pml)`, and the banner reports the real layer parameters; when disabled, behavior is byte-identical to today.

Files to Modify

- spatial/code/spatial_friction.hpp — add PML config fields to the numerics struct.
- spatial/code/spatial_friction.cpp — parse them from [numerics] TOML.
- drivers/spatial_dyn_driver.cpp — construct PMLayer, call SetPML, extend the banner, add CLI overrides.

Detailed Requirements

1. **Config fields** (in the same struct as use_pml, spatial_friction.hpp:97), with defaults that mean “derive”:

```
bool    use_pml           = false;
real_t  pml_thickness_m   = -1.0;    // <0 ⇒ derive = pml_cells * lc_far (see req 4)
real_t  pml_target_R      = 1e-3;
int     pml_cells         = 4;       // far-field cells across the PML when thickness derived
bool    pml_damp_bottom   = true;    // z-min absorbed
bool    pml_damp_top      = false;   // z-max = free surface ⇒ MUST stay false for SAFS
```

Parse in spatial_friction.cpp next to :796 via toml_bool/toml_real/toml_int with these defaults.

2. **CLI overrides** in the driver (alongside --pml, driver:532): --pml-thickness <m>, --pml-target-R <r>, --pml-cells <n>, --pml-damp-bottom {0,1}, --pml-damp-top {0,1}. Use the existing GetRealArg/GetIntArg/HasFlag helpers. CLI overrides config (same pattern as cli_pml at :624).
3. **Construct the layer after the bbox is known.** The reflection-warning block (driver:954-979) already computes lo,hi via pmesh.GetBoundingBox(lo,hi,1) and cp. Refactor so lo/hi/cp are available after that block, then (only when cfg.numerics.use_pml):

```
// thickness: explicit, else pml_cells * lc_far_estimate
real_t L_pml = cfg.numerics.pml_thickness_m > 0.0
    ? cfg.numerics.pml_thickness_m
    : cfg.numerics.pml_cells * lc_far;    // lc_far from req 4
int face_mask = PMLayer::FaceXLo|PMLayer::FaceXHi
    | PMLayer::FaceYLo|PMLayer::FaceYHi;
if (cfg.numerics.pml_damp_bottom) face_mask |= PMLayer::FaceZLo;
if (cfg.numerics.pml_damp_top)    face_mask |= PMLayer::FaceZHi;    // false for SAFS
pml = std::make_unique<PMLayer>(lo, hi, L_pml, cp,
    cfg.numerics.pml_target_R, /*dirs ignored*/7, face_mask);
wave.SetPML(pml.get());    // pml is a std::unique_ptr<PMLayer> with time-loop lifetime
```

The std::unique_ptr<PMLayer> pml; must be declared in the enclosing scope (outlives wave.Mult/the time loop). wave.SetAbsorbingBackground is already called at :1325 — ensure SetPML is called **after** the wave operator and background are set up and **before** the time loop. Add MFEM_VERIFY(wave.GetAbsorbingBackground() != nullptr, ...) guard so PML cannot run without Q_bg (otherwise ApplyPMLDamping asserts at runtime).

4. **lc_far source for the derived thickness.** Prefer an explicit cfg.mesh.lc_far_m if present;

otherwise estimate from the mesh as the max edge length in the far field, or fall back to a documented constant read from config. Do **not** hardcode 3000 in C++ — read `lc_far` from [mesh] (add `lc_far_m` to the mesh config if absent) so the default thickness tracks the mesh. If neither is available, require `--pml-thickness` explicitly and abort with a clear message.

5. **Banner** (driver:757): replace the bare use `pml: yes/no` with, when enabled:

```
use pml:          yes (L=12.0 km, R0=1e-3, faces=x±,y±,z-bottom; free surface z=0 undamped)
PML inner edges:  x[329.8,657.3] y[3657.3,3874.0] z[-26.6,-] km
PML d_max:        5.18 s^-1    (1/d_max = 0.19 s)
PML/fault clearance: x≥35 y≥34 bottom 13 km
```

Compute and print the inner edges and clearances from `lo/hi`, `L_pml`, and the fault bbox if available (else omit clearance). This makes “is the PML eating the fault?” answerable from the log.

6. **Suppress the reflection warning correctly** (driver:973): the `!cfg.numerics.use_pml` guard already does this; leave as-is.

Interfaces

- New numerics fields (req 1); new CLI flags (req 2).
- `std::unique_ptr<PMLayer>` in the driver scope; `wave.SetPML(pml.get())`.

Edge Cases to Handle

- `use_pml=false` → no `PMLayer` constructed, `SetPML` never called → byte-identical to today.
- `pml_damp_top=true` on SAFS → allowed but log a prominent WARNING (“damping the $z=0$ free surface — unphysical for a half-space; intended only for a full-space test”).
- $L_{pml} \geq \text{buffer}$ (PML would reach the fault) → abort with the computed clearance and a suggestion (reduce `pml_cells/thickness` or remesh with larger pads). Use the fault bbox if known; otherwise warn that clearance is unchecked.
- Restart (`--restart`): PML is stateless (no memory variables), so a restart simply re-constructs the same layer — verify the bbox/cp are recomputed identically post-restart.

Acceptance Criteria

- ☐ `use_pml=false`: a short SAFS smoke is byte-identical to the current binary (diff the [DIAG]/step lines for the first 200 steps).
- ☐ `--pml`: banner prints real `L_pml`, `R0`, `d_max`, inner edges, clearances; `wave.GetPML() != nullptr`.
- ☐ A 2–4 rank local smoke with `--pml` on a small box runs without the `has_bulk_bg_assert` and without NaN through nucleation.
- ☐ make `seas_spatial_dyn_driver` clean; make test green.

Dependencies

- Depends on: Phase 1. Required by: Phase 3.

1.8 Phase 3 – Validate PML effectiveness on the SAFS problem

Goal

Quantitatively demonstrate the PML removes the post-7 s reflection contamination, and use it to settle whether the LSW $t \approx 28$ s $V_{\max}$ departure is reflection-driven or a genuine friction instability.

Files to Create

- `safs/project_7.0_alternative/spatial/code/scripts/pml_reflection_metrics.py` — post-processing metric tool (reads the fault `.vtkhdf` + the `[DIAG]/[DIAG-SIGN]` log lines).
- Three sbatch variants under `jobs/safs/` (or `SAFS_*` env overrides on the existing dev sbatch): `pml_on`, `pml_off` (baseline), `bigbox_ref` (no PML, deeper/wider box).

Detailed Requirements (effectiveness checks, in increasing strength)

1. **A/B $V_{\max}(t)$ overlay (cheapest, directly answers the user’s question).** Run identical configs `--pml` vs no-PML to the same `tfinal` (≥ 34 s to cover the observed departure; ideally to ~ 40 – 50 s).
 - **Decision rule:** if the $t \approx 28$ s $V_{\max}$ departure **disappears or is strongly delayed** with PML $\rightarrow$ it was **reflection-driven**. If it **persists** under PML $\rightarrow$ it is a **genuine friction/LSW instability** (then the cap/mesh/flux thread, not the boundary, is the culprit). Either outcome is a definitive result.
2. **Reflection-free-window extension.** Without PML the fault fields diverge from the “truth” after $t_{\text{reflect}} \approx 6.94$ s. With PML, the fault `[DIAG-SIGN]` `sigma_n_min(t)` and $V_{\max}(t)$ should remain smooth past 7 s (no step/kink at the reflection arrival). Metric: max relative difference of $V_{\max}(t)$ between PML and a **big-box reference** over $[0, t_{\text{reflect_bigbox}}]$.
3. **Big-box reference (gold standard).** Remesh with `--pad-x/--pad-y 100000 --pad-bottom 60000` ($\approx$ doubles the smallest dim $\rightarrow t_{\text{reflect}} \approx 14$ – 17 s), run **no PML**. Over $[0, \sim 14$ s] the big-box run is reflection-free truth. **Accept PML** if $\max_t |V_{\max}^{\text{PML}} - V_{\max}^{\text{bigbox}}| / V_{\max}^{\text{bigbox}} < 5\%$ over that window (mirrors the unit-test 5% oblique bar at problem scale).
4. **Near-boundary seismogram (point check).** Add 2–3 receiver points $\sim$ one wavelength inside the PML inner edge (one mid-side, one near the bottom). With effective PML the velocity trace shows the outgoing pulse with **no reflected arrival**. The metric tool computes the ratio of the reflected-arrival peak to the incident peak (target $\leq \sim 0$ – 1%).
5. **Interior energy monitor.** Total elastic energy integrated over the **interior only** (exclude the PML shell) should rise during rupture then **decay monotonically** as radiation exits — no spurious re-injection bump at $t \approx 7$ s (the no-PML signature). Reuse the `EnergyDensity` integrand from `test_pml.cpp`.

Acceptance Criteria

- ☐ PML-vs-no-PML $V_{\max}(t)$ overlay produced; the $t \approx 28$ s departure is classified (reflection-driven vs intrinsic) with the decision rule in req 1.
- ☐ max relative $V_{\max}$ difference (PML vs big-box) $< 5\%$ over the big-box reflection-free window.
- ☐ Near-boundary reflected/incident peak ratio $\leq \sim 1\%$.
- ☐ Interior energy decays monotonically after the main radiation phase (no $t \approx 7$ s re-injection bump).
- ☐ PML run reaches the same step rate ($\pm 20\%$) as no-PML (the per-QP damping loop is cheap;

confirm no pathological slowdown).

Dependencies

- Depends on: Phase 2. Required by: nothing (Phase 4 optional).
-

1.9 Phase 4 (optional) – Production integration & remeshed deep-box variant

Goal

Promote the validated PML to the production sbatches and, if Phase 3 flagged the bottom buffer as too thin, ship a deeper-bottom mesh.

Detailed Requirements

1. Add `--pml (+ [numerics] use_pml=true, pml_cells, pml_target_R)` to the normal-queue parents (`spatial_dyn*_normal_48hr_safs.sbatch`) and the dev smoke sbatches, with a banner-grep guard (“PML: ACTIVE (L=... faces=...)”) mirroring the existing floor-banner guard.
2. **Only if Phase-3 req-3/req-4 fail at the bottom:** remesh via `run_z0cut_meshing.py --pad-bottom 40000` (deepen 15 km) so the bottom PML clears the sub-fault region by ≥ 25 km; regenerate the triq mesh; rerun Phase-3 metrics.
3. Update `scripts/estimate_output_size.py` only if PML changes field registration (it does not — PML adds no output fields).

Acceptance Criteria

- ☐ Production sbatch banners confirm PML active; a restart-chain dev run resumes with PML and stays bounded.
- ☐ (If remeshed) bottom clearance ≥ 25 km; Phase-3 metrics pass.

Dependencies

- Depends on: Phase 3.
-

1.10 Testing Strategy

- **Phase 1:** unit tests in `test_pml.cpp` — keep the 4 existing (regression for the default symmetric mask), add the 2 free-surface tests. `make test`.
- **Phase 2:** byte-identical no-PML smoke (diff first 200 step lines vs current binary); local 2–4 rank `--pml smoke` (no assert, no NaN through nucleation). `Build + make test`.
- **Phase 3:** problem-scale validation via the 5 metrics above; the big-box reference (req 3) is the correctness oracle, the unit tests are the component oracle.
- **Numerical oracles:** unit tests use analytical plane-wave eigenvectors (already in `test_pml.cpp`); the SAFS validation uses the big-box run as the reflection-free reference and energy monotonicity as a physical invariant.

1.11 Risk Assessment

- **R1 — ADER corrector vs Mult path.** PML in the predictor is intentionally skipped (`wave_operator.hpp:427`); the SAFS run uses ADER (order 2), so the **corrector** (`wave_operator.inl:5438`)

must be the active path. *Detect*: a `--pml` run must show interior energy decay; if PML appears inert, the ADER path isn't calling the damping — instrument the corrector branch with a one-time rank-0 “PML corrector active” print.

- **R2 — Free-surface damping (the main correctness trap).** If FaceZHi is ever enabled on SAFS, the PML kills the free surface → wrong surface waves / wrong rupture. *Detect*: Phase-1 `TestPMLFreeSurfaceTopUndamped` + the Phase-2 banner explicitly listing damped faces + the `pml_damp_top` WARNING.
- **R3 — PML reaching the fault.** `L_pml` larger than the buffer (esp. the 13 km bottom) damps the seismogenic zone → artificially arrests slip. *Detect*: Phase-2 clearance abort + banner clearance line; Phase-3 big-box agreement.
- **R4 — Low-frequency / surface-wave leakage.** 12–15 km is only ~ 0.5 long-period Rayleigh wavelengths; very-long-period energy may leak. *Detect*: near-boundary seismogram (req 4); mitigate by thickening x/y PML or relying on the 35–50 km side buffer. Not expected to affect the body-wave-dominated $t \approx 7$ s contamination.
- **R5 — Cost.** The per-QP damping loop runs every stage over PML-shell elements only; with `lc_far=3000` the shell is a thin sliver of the mesh, so cost should be $<$ a few %. *Detect*: Phase-3 step-rate comparison.
- **R6 — Restart determinism.** PML is stateless, but the `bbox/cp` must be recomputed identically on restart. *Detect*: a checkpoint→restart smoke that reproduces the pre-checkpoint `V_max` to round-off.
- **Tricky existing code:** `wave_operator.inl:5438–5490` (corrector damps `I - dt·Q_bg`, not `Q - Q_bg`) — do not “simplify” it to match the `Mult` form; the `dt·Q_bg` target is correct for the time-integrated ADER state.

1.12 Summary of file touches (for the implementer)

Phase	File	Change
1	<code>dynamic/pml_layer.hpp</code>	add half-face mask enum + optional ctor arg + <code>GetFaceMask</code>
1	<code>dynamic/pml_layer.cpp</code>	honor <code>face_mask_in</code> <code>ComputeDamping</code> ; derive mask from <code>dirs</code> when <code>-1</code>
1	<code>tests/unit/test_pml.cpp</code>	+2 free-surface tests; keep 4 existing
2	<code>spatial/code/spatial_friction.hpp</code>	drop PML numerics fields
2	<code>spatial/code/spatial_friction.cpp</code>	pass them from <code>[numerics]</code>
2	<code>drivers/spatial_dyn_driver.cpp</code>	construct <code>PMLayer</code> from <code>bbox+config</code> , <code>SetPML</code> , CLI flags, banner, clearance guard
3	<code>safs/.../scripts/pml_reflection_metrics_tool</code>	new metrics tool
3	<code>jobs/safs/*pml*</code>	A/B + big-box sbatch variants

Phase	File	Change
4	production sbatches / mesher	optional promotion + deep-bottom remesh

Bottom line on the two questions you asked directly: 1. **How large to extend the domain?** You don't — the existing 50/50/25 km pads host a 12 km (default) or 15 km PML on the five absorbing faces with ≥ 10 km fault clearance. A bigger box would need ~ 300 km pads to do the same job without PML. 2. **How to check effectiveness?** Phase-3's five metrics, anchored by the **big-box reference** ($< 5\%$ $V_{\max}$ agreement) and the **near-boundary reflected/incident ratio** ($\leq \sim 1\%$); the **PML-vs-no-PML $V_{\max}(t)$ overlay** simultaneously settles whether the $t \approx 28$ s LSW departure is reflection-driven or intrinsic.